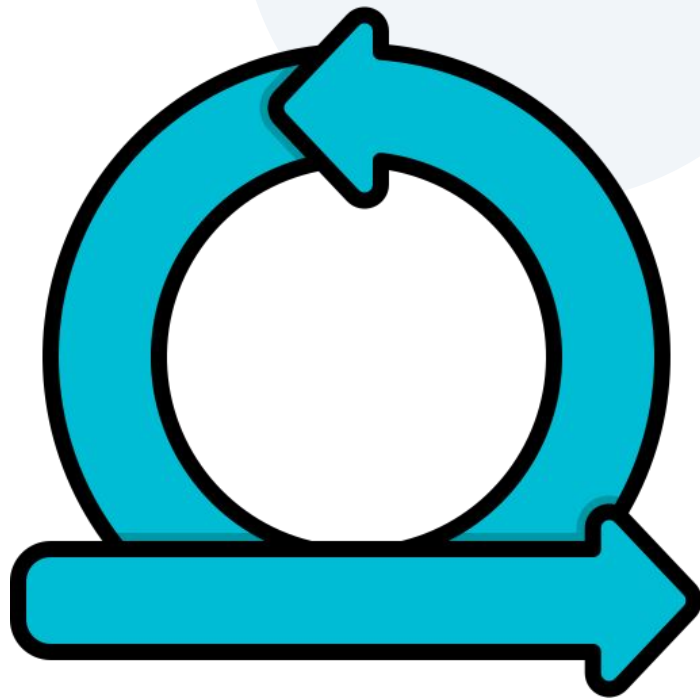


Testes e Desenho RTL

Sprint 3

Rafael Oliva Soares, Derick Maciel Muniz, Luis Felipe Caleal Macalli



Objetivos

Benchmark

O algoritmo Perceptron para garantir precisão e desempenho nos testes iniciais.

Elaborar

Desenhos da arquitetura em RTL, definindo a estrutura lógica do hardware.

Validação do Algoritmo

Garantia de Colisão

Mapeamento Forçado no Set 0

Para testar a inteligência do preditor, forçamos os endereços a colidirem intencionalmente no mesmo conjunto de cache:

Endereço de Acesso: Calculado como o produto de conjuntos pelo tamanho do bloco.

Endereços Colidentes: `addr_hot`, `addr_1` e `addr_2` são múltiplos exatos de endereços.

Isto obriga a cache a realizar substituições com base na decisão de pesos do Perceptron.

Validação Funcional do Algoritmo

FASE 1: AQUECIMENTO E PREENCHIMENTO DO SET 0

Dois endereços colidentes realizam acessos iniciais compulsórios de inicialização (**MISS**), ocupando toda a associatividade do conjunto físico.

Via 0 (Bloco Quente)

Endereço Físico: 0x00000000 PC: 0x00400004

TAG: 0 Score Inicial: **u** = 0

Via 1

Endereço Físico: 0x00000800 PC: 0x00400000

TAG: 1 Score Inicial: **u** = 0



GHR GLOBAL (HISTÓRICO)

0x00 (00000000)



TABELA DE PESOS (BIAS + PESOS GHR)

[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Validação Funcional do Algoritmo

FASE 2: REFORÇO E TREINAMENTO DE PESOS (LOOPS)

A simulação realiza 10 **HITs** consecutivos no Bloco da Via 0. A rede neural identifica o padrão altamente repetitivo e altera estavelmente os coeficientes lineares.

Via 0 (Bloco Altamente Ativo)

Endereço Físico: 0x00000000 PC de Origem: 0x00400004

TAG: 0 Score: **u = +18**

Via 1 (Inalterada)

VÁLIDO

Endereço Físico: 0x00000800 PC de Origem: 0x00400000

TAG: 1 Score: u = 0



GHR

0xFF (11111111)

Validação Funcional do Algoritmo

FASE 3: RESOLUÇÃO DE CONFLITO E VEREDITO

O carregamento do terceiro endereço (**0x00001000**) força um descarte. A IA protege o bloco com score mais alto e expulsa o de menor score.

✓ Via 0 (PRESERVADA)

Endereço Físico 0x00000000 PC de Origem 0x00400004

TAG: 0 Score: u = +2

⚠ Via 1 (VÍTIMA DA EXPULSÃO)

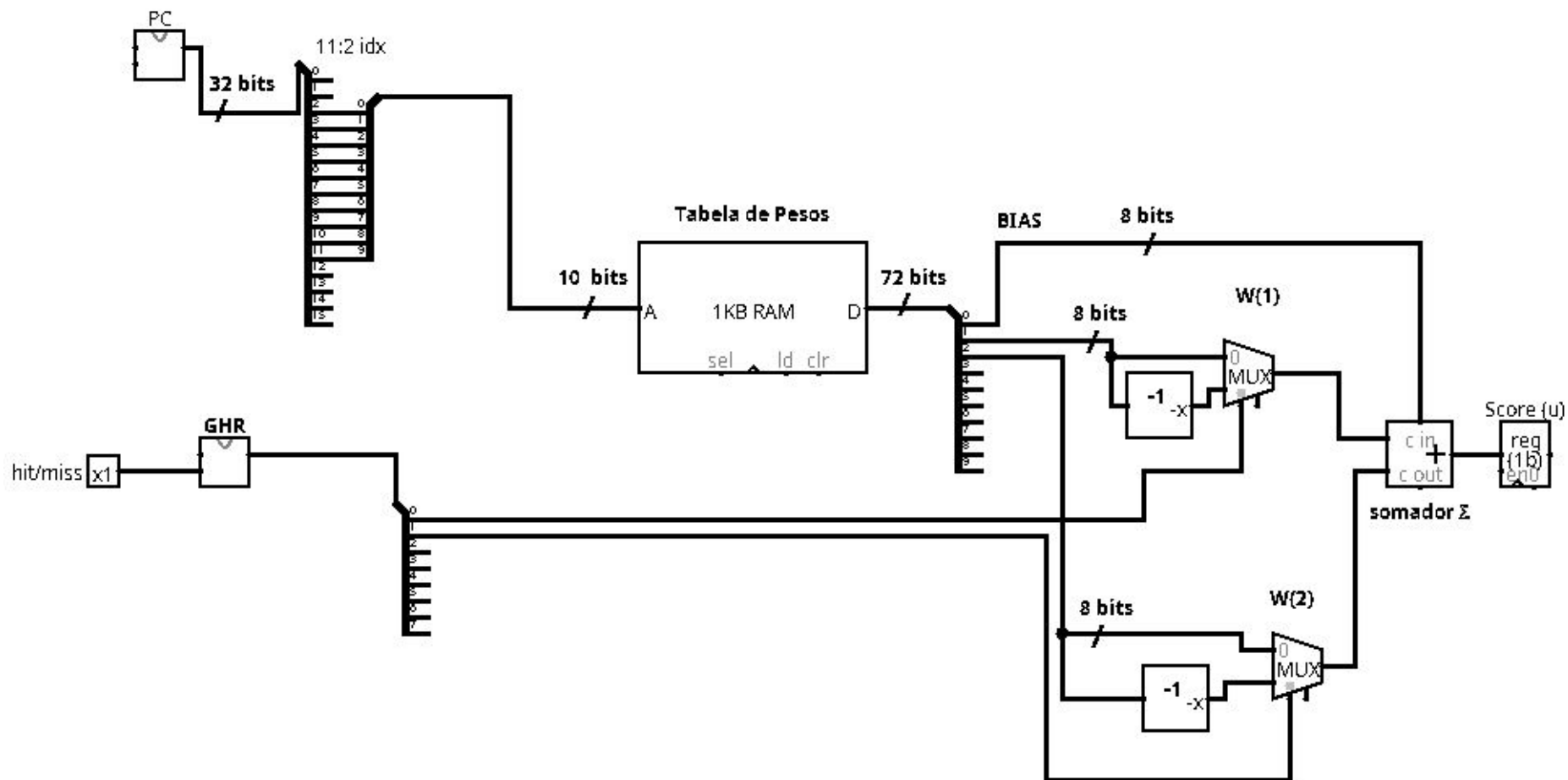
SUBSTITUÍDA

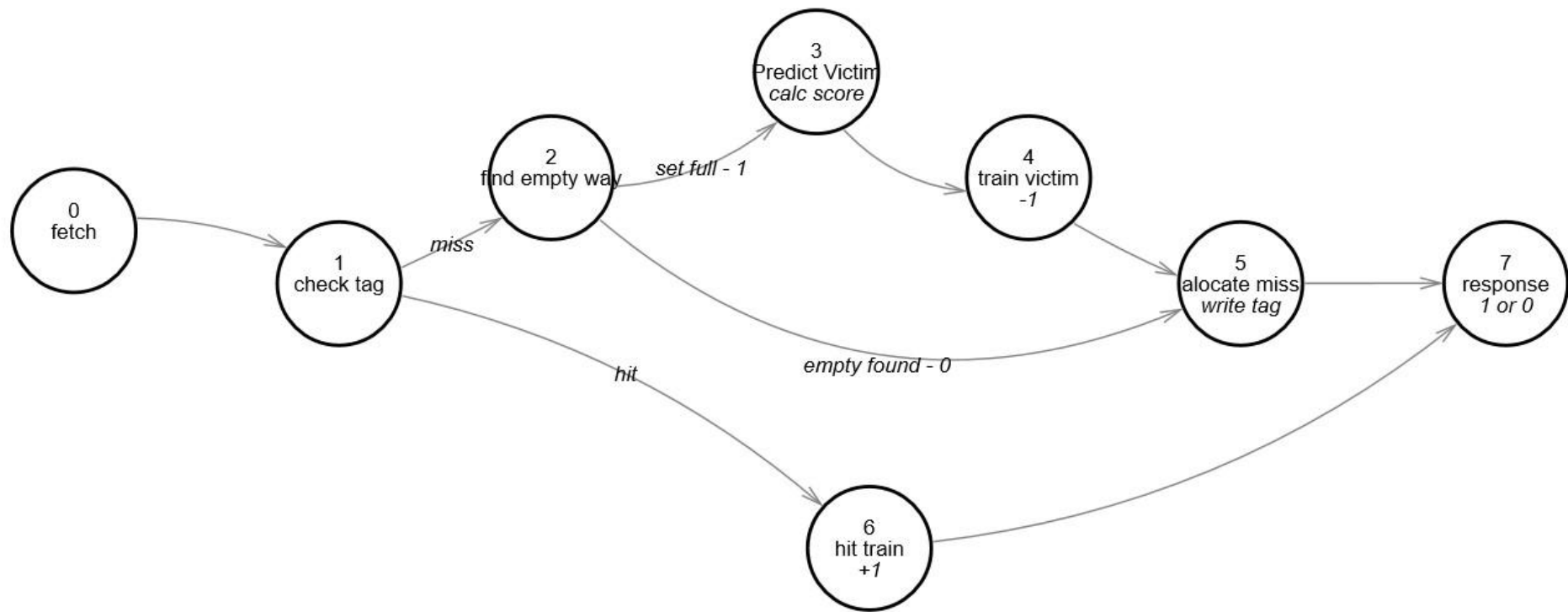
Novo Endereço **0x00001000** PC Conflitante 0x00400000

Nova TAG: 2 Score: u = -7

GHR RESULTANTE
(11111110)

Perceptron - RTL





Implementação de Histerese

O problema

Um limiar estático (Threshold) reage de forma muito agressiva a ruídos momentâneos na cache. Isso causa oscilações nos pesos e destrói padrões estáveis recém-aprendidos.

Implementação de Histerese

O limiar deixa de ser fixo e torna-se adaptativo e dinâmico.

Em caso de ACERTO:

O limiar diminui Evita treinos desnecessários e estabiliza os pesos que estão funcionando.

Em caso de ERRO:

O limiar aumenta torna o treino rigoroso para corrigir o erro o mais rápido possível.

Histerese - Implementação no código

Cache.h

Adicionado o campo **int hysteresis**; na estrutura principal da cache, encapsulando o parâmetro de sintonia.

Cache.c

A função **cache_init** foi modificada para inicializar o parâmetro. A lógica de treino (perceptron_train) calcula o effective_threshold dinamicamente antes de aplicar a saturação

Main.c

Centralização da variável **default_hysteresis = 2**, injetando-a em todos os testes interativos e no Grid Automatizado

Histerese - Implementação no código

Cache.h

Adicionado o campo **int hysteresis**; na estrutura principal da cache, encapsulando o parâmetro de sintonia.

Cache.c

A função **cache_init** foi modificada para inicializar o parâmetro. A lógica de treino (perceptron_train) calcula o effective_threshold dinamicamente antes de aplicar a saturação

Main.c

Centralização da variável **default_hysteresis = 2**, injetando-a em todos os testes interativos e no Grid Automatizado

Histerese - Avaliação de Benchmarks

Ganhos Críticos:

No cenário **Cyclic Thrashing**, onde a política clássica **LRU** colapsa totalmente alcançando **0.00%** de acertos, a inércia do **AIRA** reteve blocos importantes e garantimos um ganho real de **6.34%**.

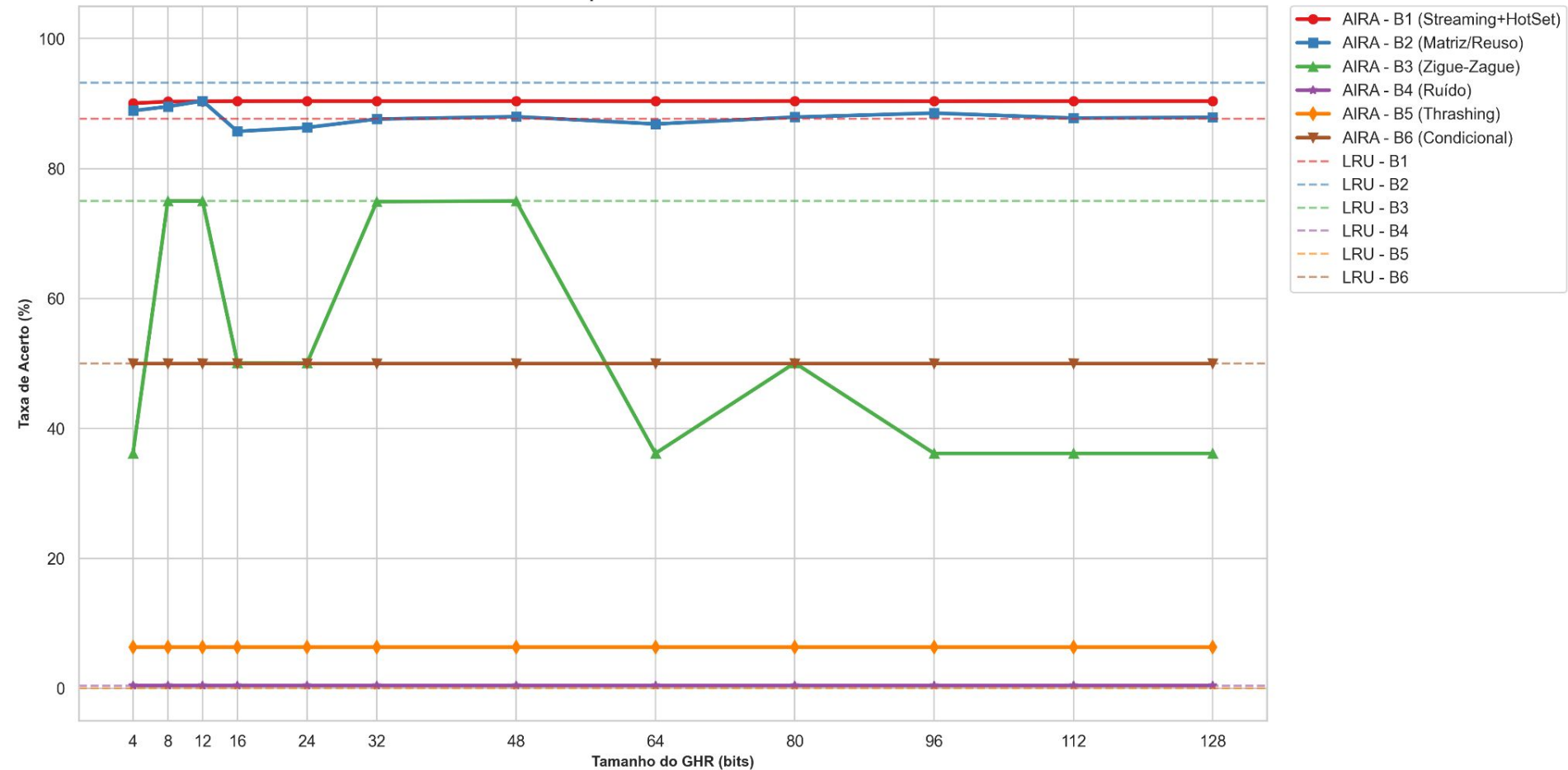
Streaming:

Registramos um impacto positivo de **+2.61%** sobre o **LRU**, provando a mitigação eficaz de poluição de dados.

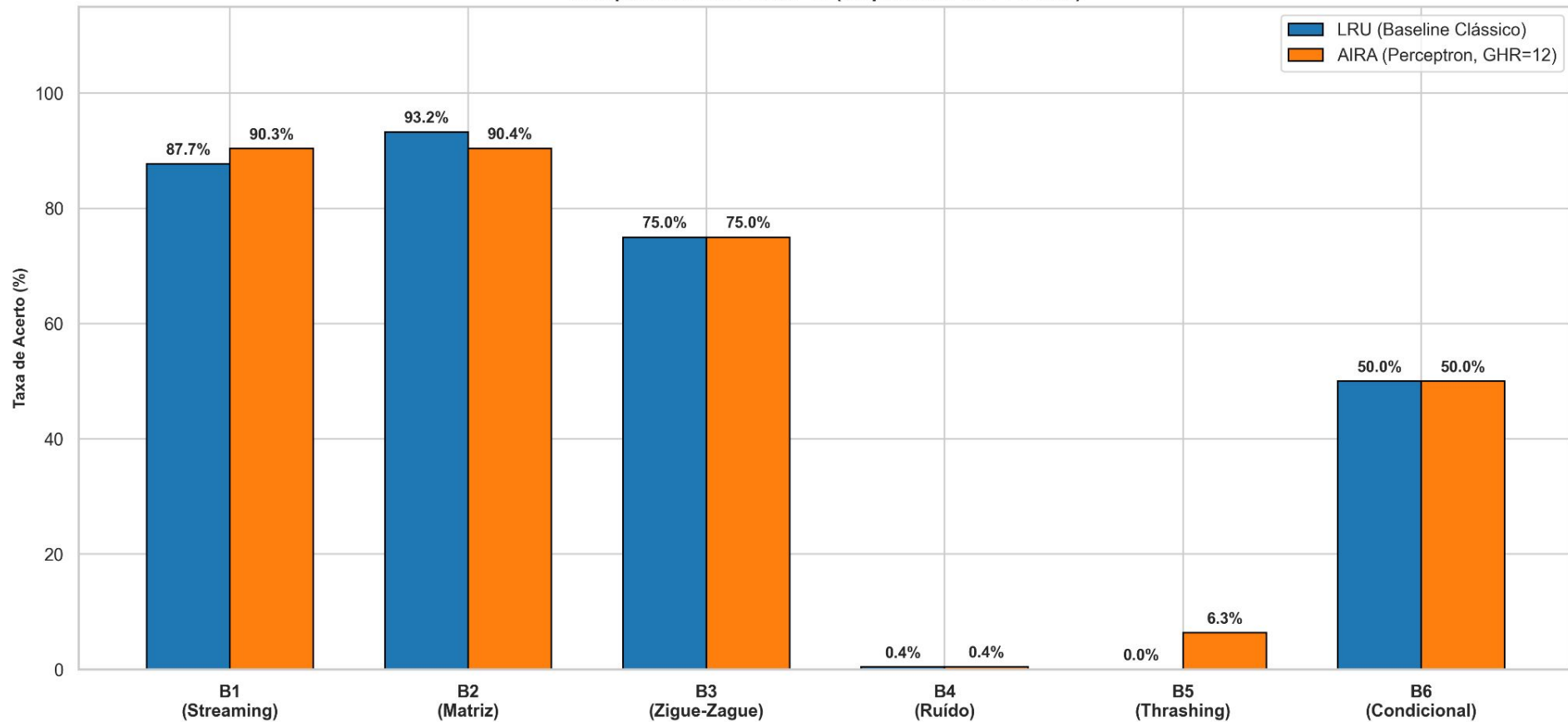
Hash Table Noise:

O empate em aproximadamente **0.4%** válida matematicamente que o modelo é preciso e não gera acertos artificiais em fluxos puramente imprevisíveis (aleatórios).

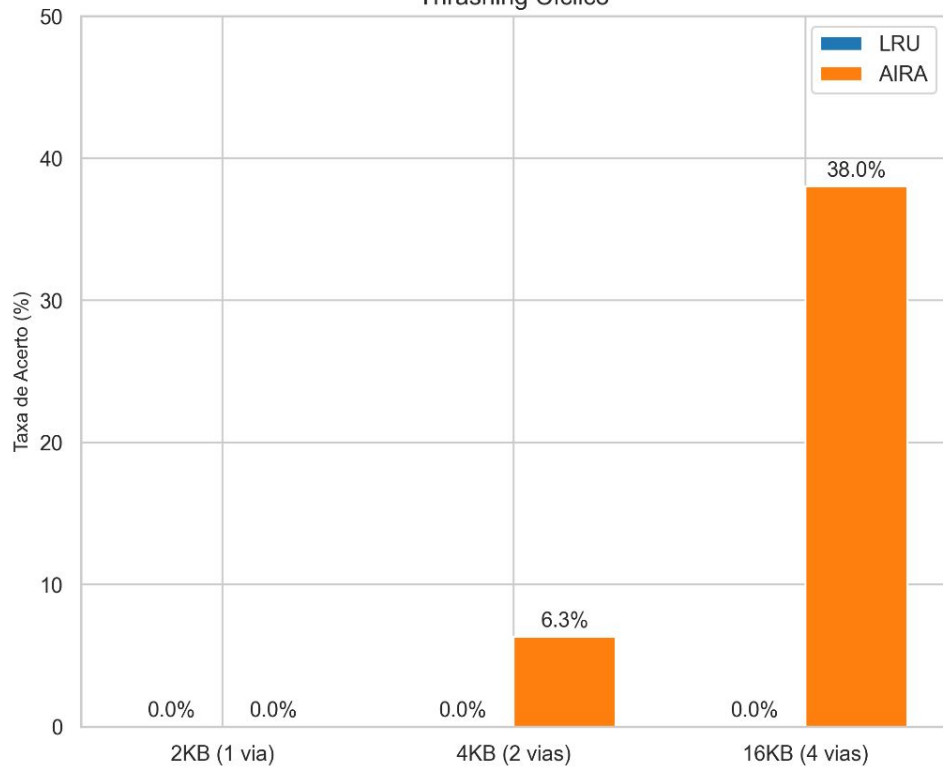
Impacto do GHR na Taxa de Acerto dos 6 Cenários
Arquitetura: 4KB / 2 Vias



LRU vs AIRA
Comparativo com GHR=12 (Arquitetura 4KB / 2 Vias)



Thrashing Cíclico



Condicional

